

刘田田 25300120181 的实验报告:

## 一、文档中要求回答的问题

1. 答: 我之前没有过相关经历。

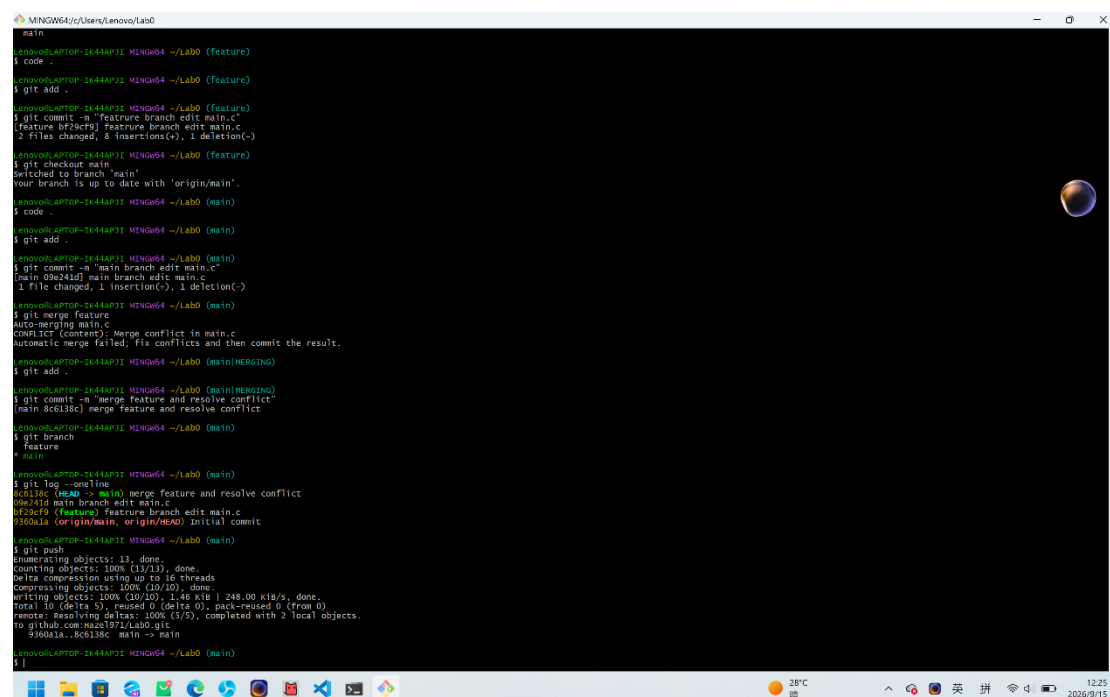
答: 暂存区可以给作者提供更多的容错空间, 方便作者在众多修改版本中挑选合适的版本作为最终版本上传, 让上传和修改更加便捷。

答: git branch 只列出本地分支, 即存放在个人电脑仓库的分支。

git branch -a 则显示全部分支, 即本地分支与远程追踪分支

3. 答: 学习 git 能让集体编程过程变得更加清晰规范, 我们能清楚知道每一个改动的来源, 每一段代码的目的, 这有助于帮助前期清晰有效的编程, 还有利于后期高效的维护工作, 如果出现问题, 还能帮助我们快速找到问题来源进行修改。

4. 答: 冲突发生的原因是两个分支上对同一文件的同一部分进行了不同的修改, git 无法判断应该保留哪个版本, 因此需要手动解决冲突。



```
main
MINGW64/C:/Users/Lenovo/Lab0
$ cdovvmlaptop-1k44apj1 MINGW64 ~/Lab0 (feature)
$ code .
$ git add .
$ git commit -m "feature branch edit main.c"
[feature bf29cf9] feature branch edit main.c
2 files changed, 8 insertions(+), 1 deletion(-)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ code .
$ git add .
$ git commit -m "main branch edit main.c"
[main 09e241d] main branch edit main.c
1 file changed, 1 insertion(+), 1 deletion(-)
$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
$ git add .
$ git commit -m "merge feature and resolve conflict"
[main 8cd338c] merge feature and resolve conflict
1 file changed, 1 insertion(+), 1 deletion(-)
$ git branch
* main
  feature
$ git log --oneline
cd338c (HEAD) merge feature and resolve conflict
09e241d main branch edit main.c
bf29cf9 (feature) feature branch edit main.c
93b0a1a (origin/main, origin/HEAD) initial commit
$ git push
Enumerating objects: 33, done.
Counting objects: 100% (33/33), done.
Delta compression using up to 16 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 1.46 KiB | 248.00 KiB/s, done.
Total 10 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 2 local objects.
to github.com:maze1971/Lab0.git
93b0a1a..8cd338c main -> main
$
```

## 二、实验步骤

1. 首先遵循网页教程在 github 模拟仓库上克隆建立了自己的仓库到本地, 并在 git 里面输入了 git branch 确保当前处于 main 主分支。

2. 在 README 中编辑文档回答文档问题, 在回答第二题时, 编辑 main.c 填写字符串后, 执行 git add\git commit 提交。

3. 第四大题的分支实验过程:

- 1) 首先使用 `git checkout -b feature` 创建并切换到 feature 分支, 修改 `main.c`, `commit` 提交
- 2) `git checkout main` 切回 main 分支, 修改 `main.c` 在 feature 中同一行的位置, 改为不同内容, 提交
- 3) 在 main 分支执行 `git merge feature`, 触发合并冲突
- 4) 在 VScode 打开 `main.c`, 删除 `<<<<< HEAD`、`=====`、`>>>>> feature` 和冲突地带不需要保存的代码, 保存文件
- 5) `git add .` 标记冲突已经解决, `git commit` 完成合并并提交。
4. 将两张冲突截图通过 `git push` 上传到 GitHub 仓库, 之后上传到实验报告。
5. 保存上传所有文件到 GitHub。